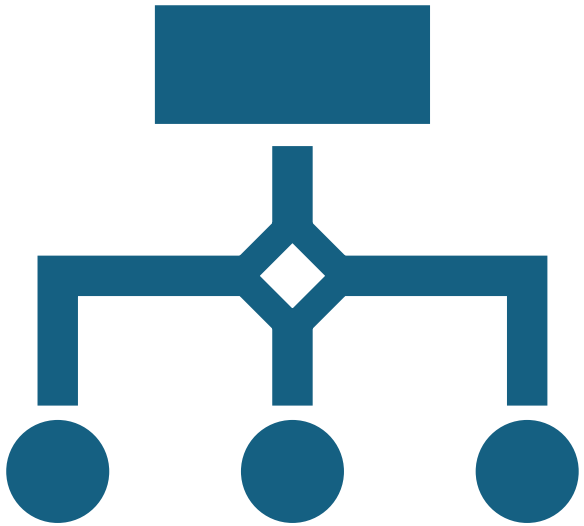
Introdução ao Git: um guia prático para iniciantes

Erick Sharlls Ramos de Pontes
erick.sharlls@ifpb.edu.br
https://github.com/ericksharlls

Git

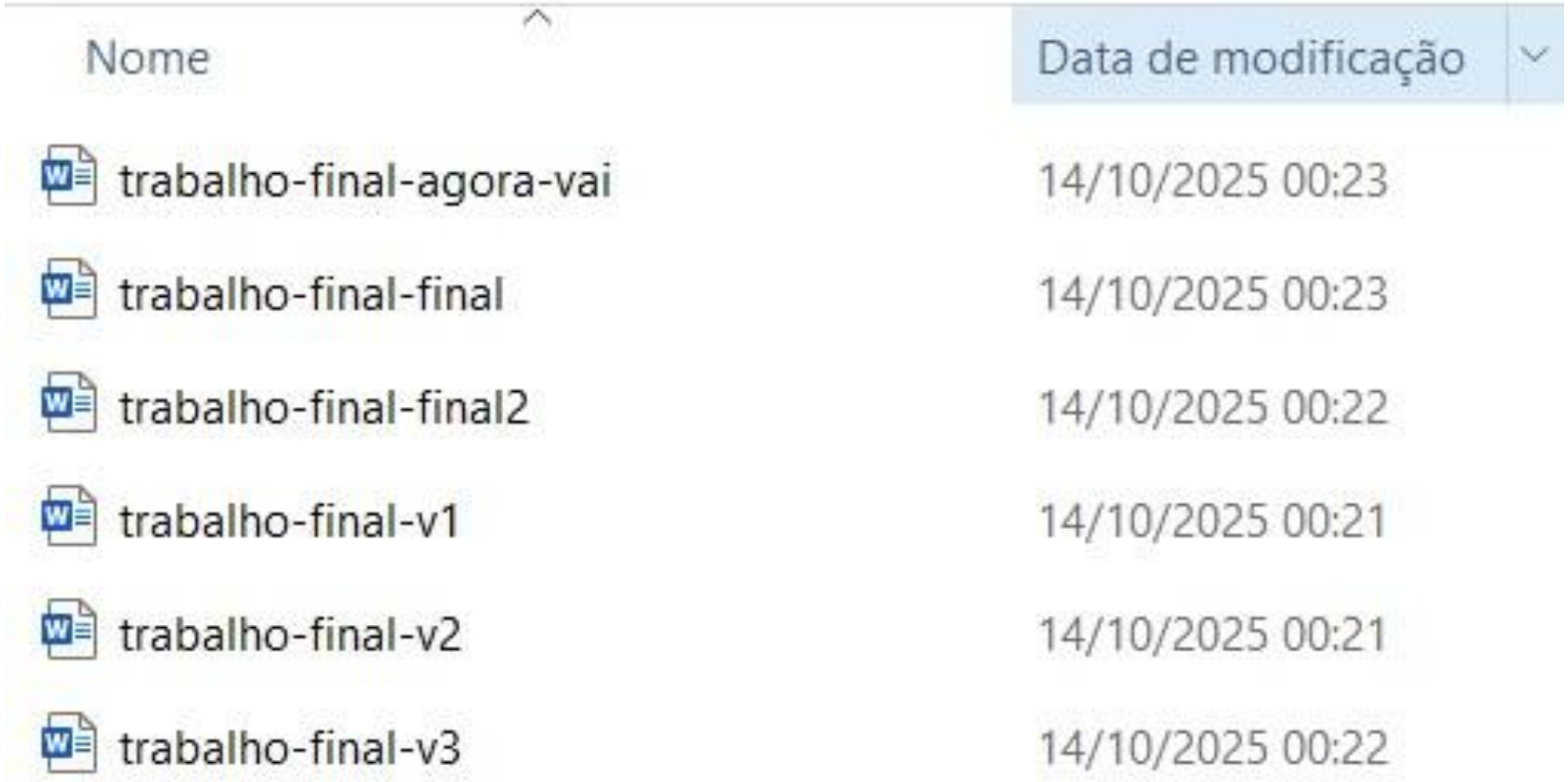
- Sistema de versionamento
- Serve para controlar versões de documentos, projetos, etc





Como assim controlar versões?

Problemática



Temos necessidade real de versionar nossos trabalhos

Manter histórico, como ele está hoje, como estava antes, etc

Então, preciso usar uma ferramenta especializada para isso...



Git





Git

É um sistema de versionamento (gerencia suas versões de trabalho) que você instala em seu computador

Ele vai permitir você manter um histórico, ver diferença entre uma versão e outra, voltar versão, trabalho em equipe, etc

Git x Github



Github

- Serviço online de hospedagem de projetos gerenciados pelo *Git*
- Você pode hospedar seus projetos de graça, você pode seguir outros usuários, outros usuários podem te seguir
- Mais de 100 milhões de desenvolvedores utilizam
- É o maior repositório de dados compartilhado do mundo
- Grandes projetos no mundo inteiro estão hospedados no *Github*

Minha página no github:

https://github.com/ericksharlls

Mas existem os concorrentes do Github

- Bitbucket - https://bitbucket.org/
- GitLab - https://gitlab.com/

Nessa oficina usaremos...



Como verificar se o Git está instalado?

- ***git --version***
- Git instalado, agora crie uma pasta do seu projeto no computador

Criando nosso repositório git

- Acessar a pasta do projeto pelo terminal
- Criar um repositório git na pasta
  - ***git init***
- Agora, a pasta agora é um projeto gerenciado pelo ***git*** e um repositório de versionamento
- Foi criada uma subpasta oculta chamada *.git*

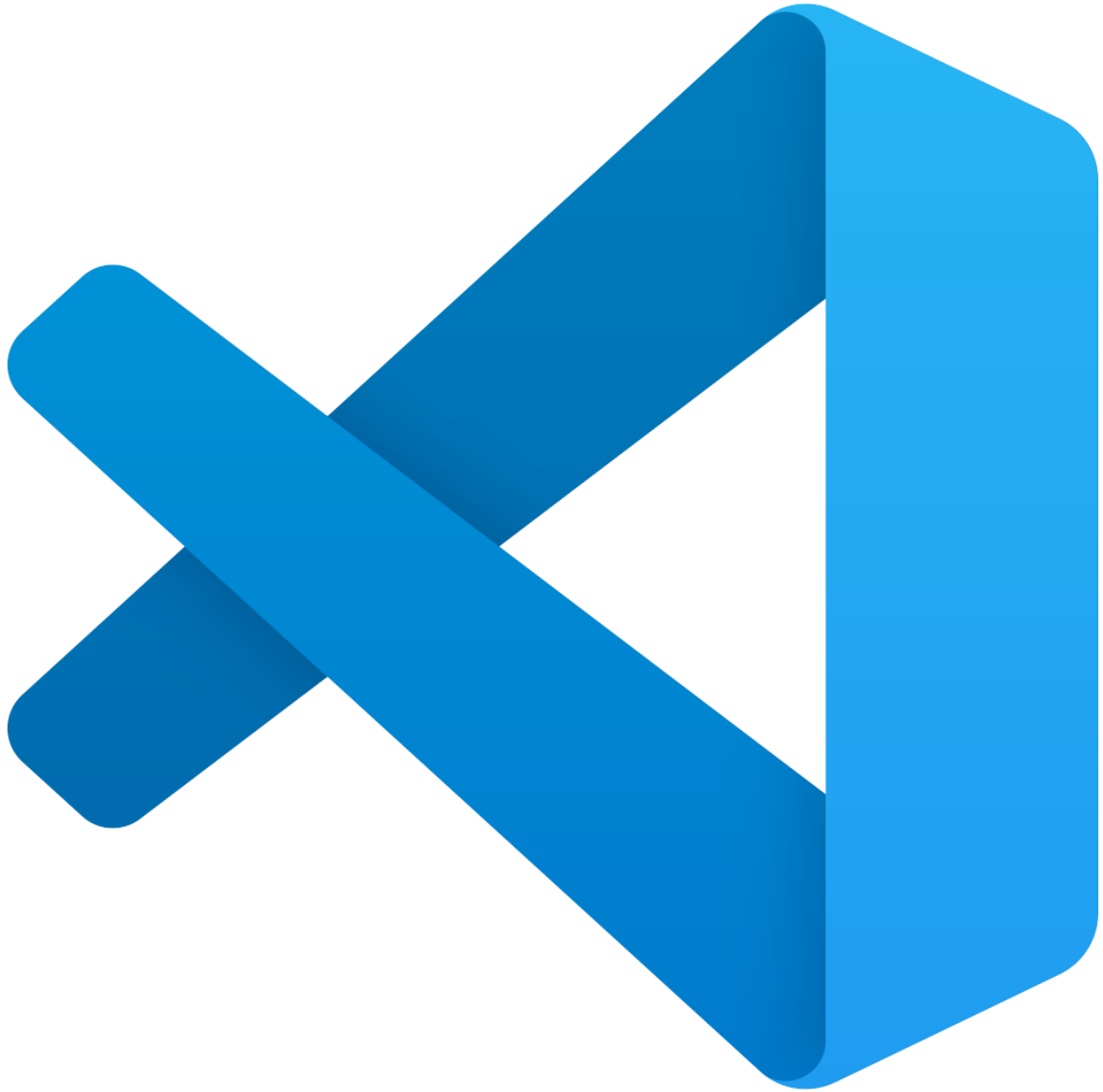
Configurando seu usuário no Git

- *git config user.name “Seu nome (não precisa ser o nome completo)”*
- *git config user.email login@dominio.com*
- *git branch -M main*

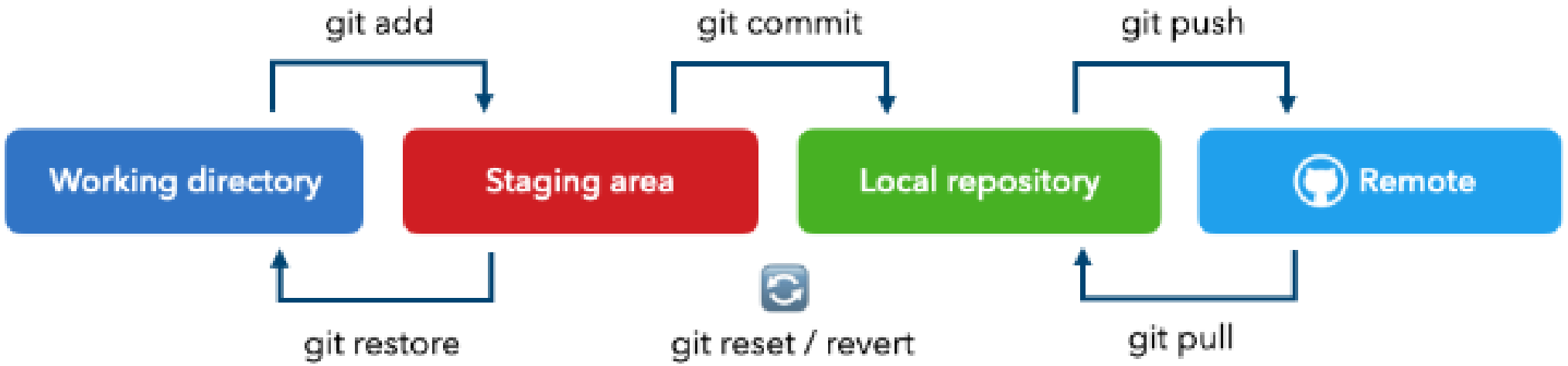
Configurando seu usuário no Git

- *git config --global user.name “Seu nome (não precisa ser o nome completo)”*
- *git config --global user.email login@dominio.com*
- *git branch -M main*
- **OBS**: use o parâmetro ***--global*** apenas no seu computador pessoal

Vamos abrir o visual studio code e criar o código dos arquivos



Estágios das mudanças no Git

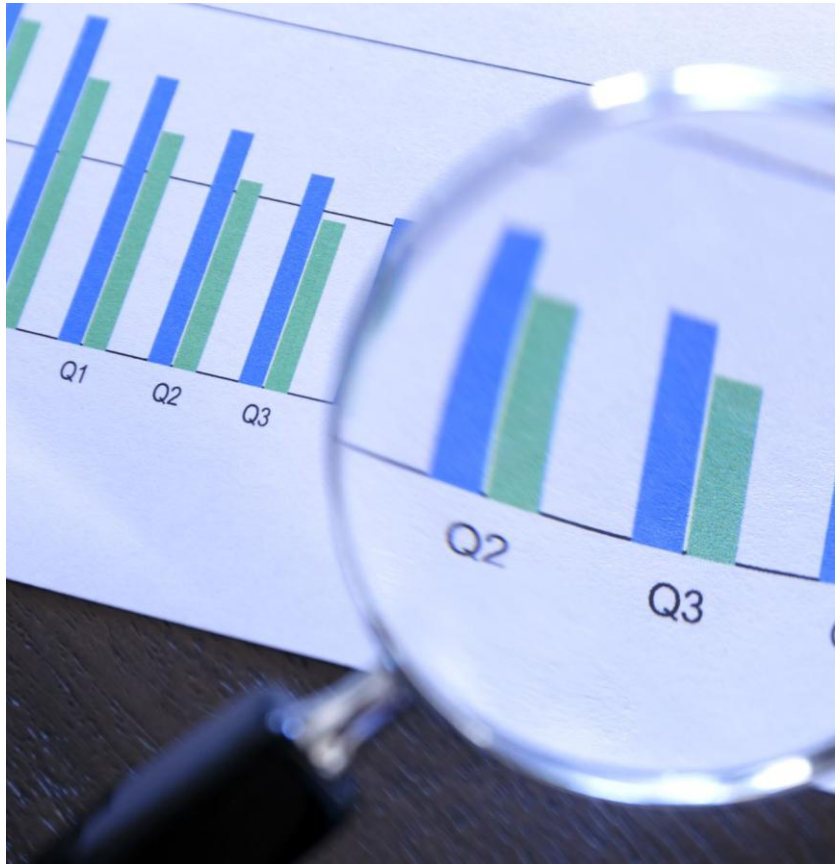


Estágios das mudanças no Git

- **Working Directory**: onde estão todos os arquivos do seu projeto
- **Staging Area**: área temporária onde você coloca os arquivos que você pretende salvar na próxima versão
  - Isso é interessante, pois você coloca os arquivos que você quiser para serem incluídos na próxima versão
- **Local Repository**: onde estão os seus arquivos confirmados (comitados) para uma nova versão

Verificando status do projeto

- Verificar em qual estágio estão as mudanças
  - *git status*

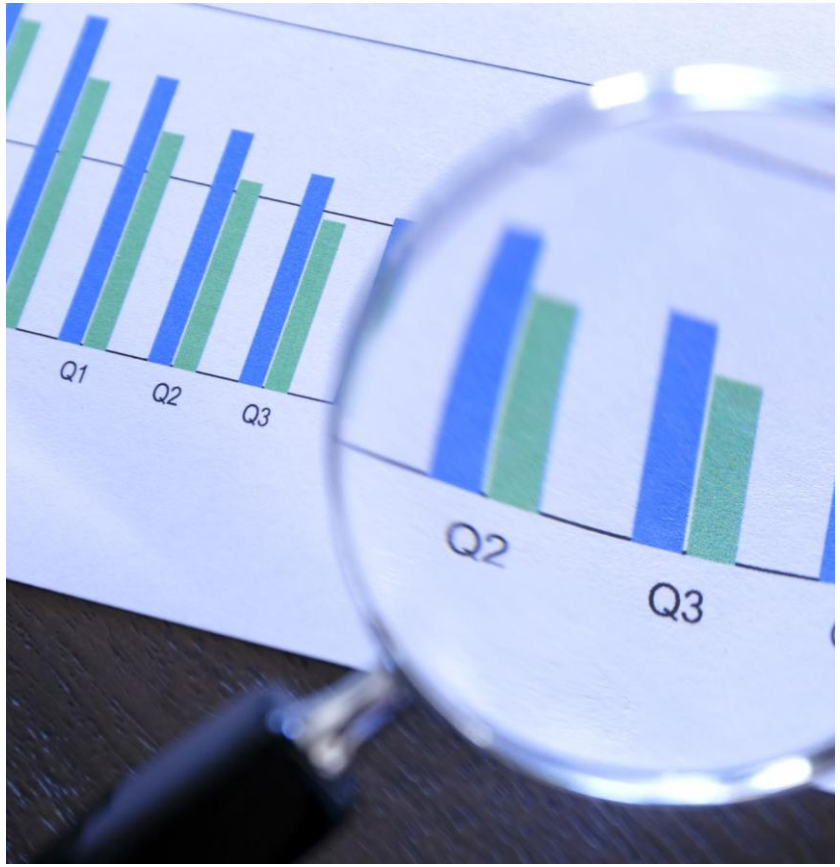


Criando a primeira versão do projeto

- Enviando para a área staging
  - *git add <nome_arquivo>* (para enviar um arquivo específico)
  - *git add .* (para enviar todos os arquivos)

Verificando status do projeto

- Verificar em qual estágio estão as mudanças
  - *git status*

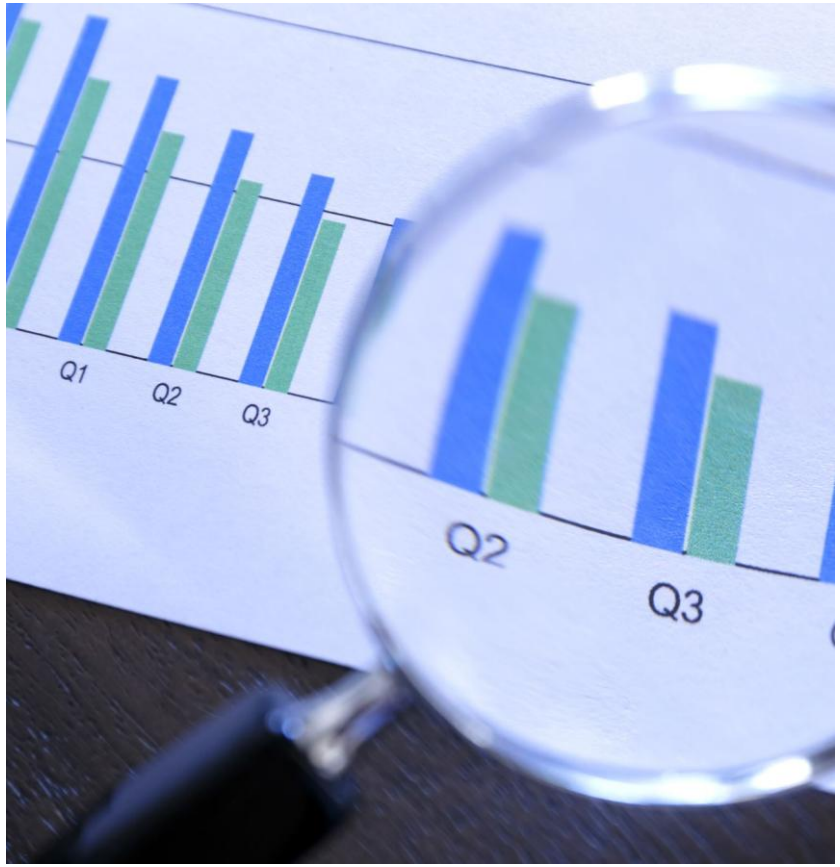


Confirmando uma nova versão (commit)

- *git commit -m “Mensagem explicativa do que está sendo feito”*

Verificando status do projeto

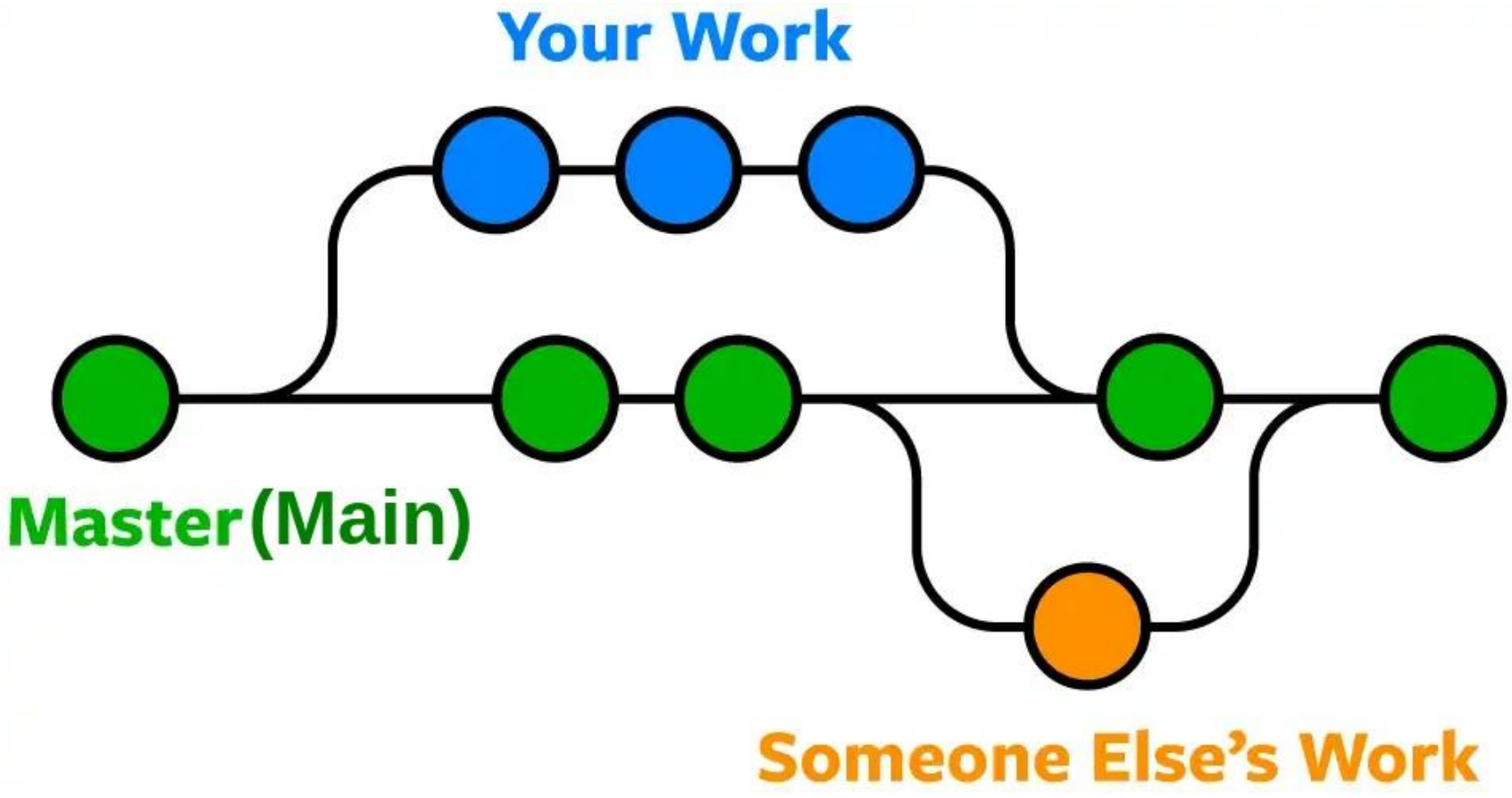
- Verificar em qual estágio estão as mudanças
  - *git status*



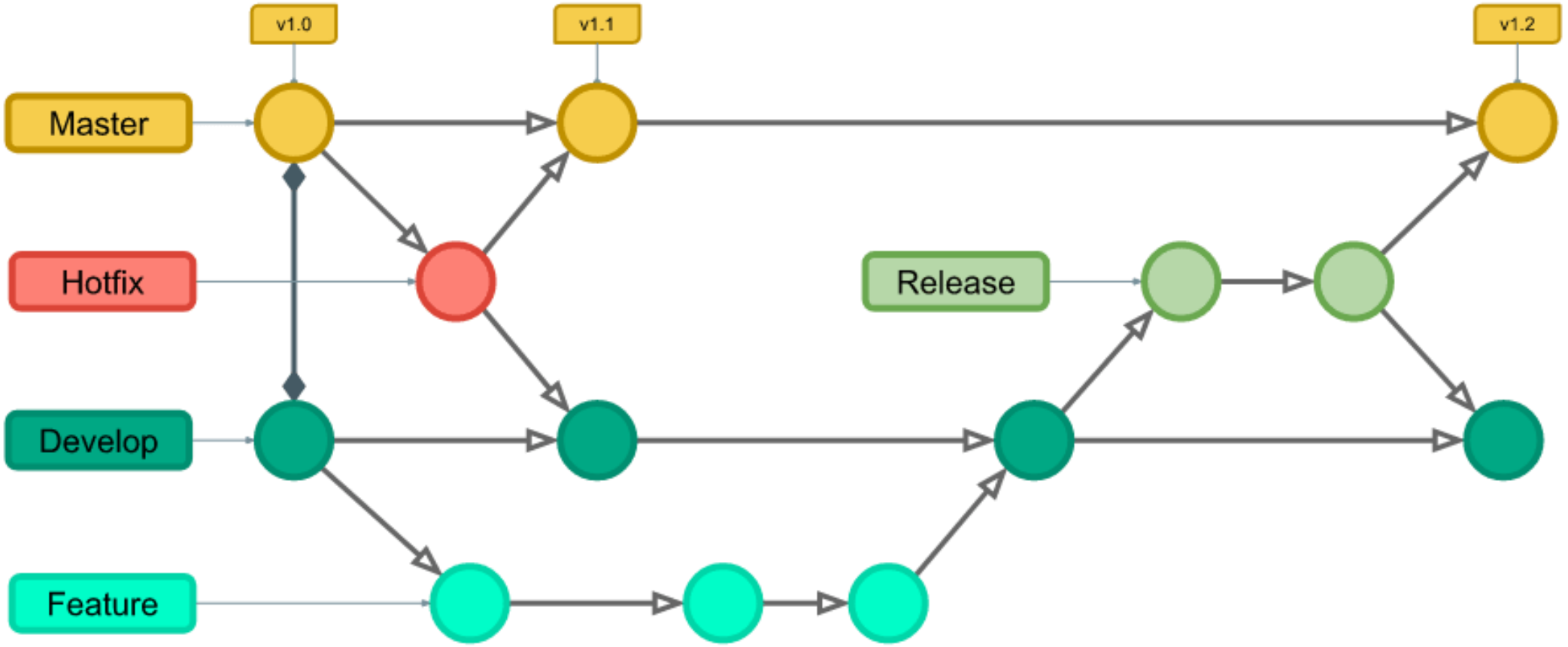
Branches

- Branches (**ramos**) são utilizados para desenvolver funcionalidades isoladas umas das outras
- O *branch* ***master/main*** é o branch padrão quando você cria um repositório
- Use outros branches para implementar novos recursos, corrigir bugs ou experimentar alterações no código sem afetar o projeto principal, e mescle-os (merge) ao *branch* ***main/master*** após a conclusão
- Branches são **caminhos** de trabalho fora do **caminho** principal

Branches



Branches - Gitflow



Criando repositório no Github

- Vamos associar o seu projeto local (no seu computador) com o projeto que está no Github
  - A associação é necessária para você enviar dados do projeto local para o remoto
- Crie um repositório público no Github
  - O Github sugere uma sequência de comandos para associar seu projeto local com o repositório remoto
  - Como forma de configuração, selecione HTTPS (também existe a opção SSH)
- Encontre um comando semelhante a esse:
  - *git remote add origin https://github.com/ericksharlls/oficina-git.git*

Enviando as alterações para o Github

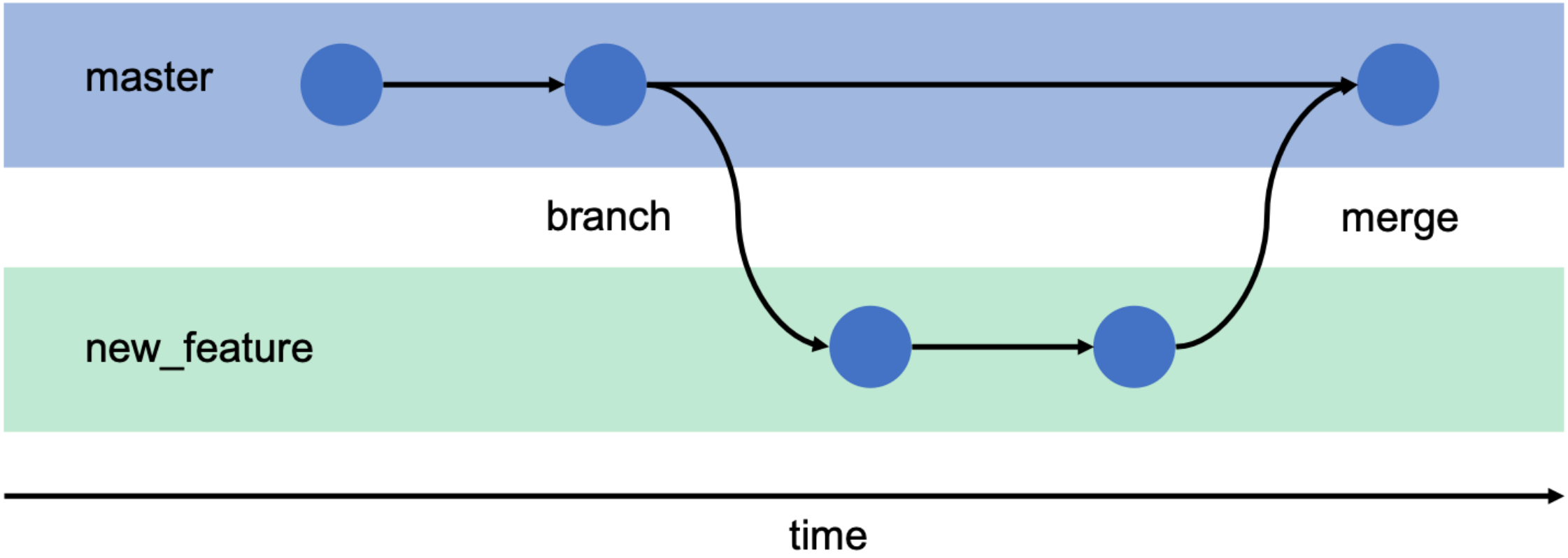
- *git push <remote> <branch>*
  - <*remote*>: Este é o nome do repositório remoto para o qual você está enviando as alterações. Por convenção, o remoto padrão é chamado de ***origin***.
  - <*branch*>: Este é o nome do *branch* local cujas alterações você deseja enviar para o repositório remoto.
- *git push origin main*
  - Este comando envia todos os commits da sua branch ***main*** local para a branch remota ***origin***

Vamos refazer o fluxo até o push para o Github?

- Altere um arquivo e depois siga o passo a passo:
  1) *git add .*
  2) *git commit -m “Mensagem explicativa do que está sendo feito”*
  3) *git push origin main*
     - *(push* envia as mudanças locais para o repositório remoto*)*

- ***git status*** a qualquer momento para verificar em qual estágio estão as mudanças
- Para descartar alterações em um arquivo no seu diretório de trabalho (Working Directory) e restaurá-lo ao estado em que estava no último *commit*, use:
  - *git restore <nome_arquivo>*
- Para remover um arquivo da área de staging, mantendo as modificações no seu diretório de trabalho, use a opção ***--staged***:
  - *git restore --staged index.html*

Trabalhando com branches



Trabalhando com branches

- Verificar em qual branch estou
  - *git branch*
- Selecionar a branch ***main***. É a partir dela que criamos outras branches
  - *git checkout main*
- Criar uma branch nova *(cria e seleciona a branch* ***pagina-clientes***)
  - *git checkout -b pagina-clientes*
- OBS: o ***checkout*** é utilizado para comutar/alternar entre branches

Trabalhando com branches

- **Passo a passo:**
  - 1) *git checkout -b pagina-clientes* (cria e seleciona a branch "pagina-clientes")
  - 2) Criar a página pagina-clientes.html
  - 3) *git add .*
  - 4) *git commit -m “Criação da página de clients”*
  - 5) *git checkout main*
  - 6) *git merge pagina-clientes* (na branch "main", faça o merge com a branch "pagina-clientes")
    - ***merge*** atualiza a branch corrente com mudanças de outra branch
  - 7) *git push origin pagina-clientes*
  - 8) *git push origin main*

Trabalhando com branches

Agora vamos aprender a resolver conflitos de merge

Trabalhando com branches

- **Simulando conflito de merge**

- **Passo a passo**:

1) git checkout main (seleciona a branch "main")
2) git checkout -b atualizacao-pagina-inicial (cria e seleciona a branch "atualizacao-pagina-inicial")
3) Alterar a página index.html
4) git add .
5) git commit -m "Atualização na página inicial"
6) Verificar como está o arquivo index.html nas duas branches (git checkout main e git checkout atualizacao-pagina-inicial)
7) git checkout main
8) Alterar a página index.html
9) git add .
10) git commit -m “Atualização na página inicial”

Trabalhando com branches

- **Resolvendo o conflito de merge**
- **Passo a passo:**
  11) *git merge atualizacao-pagina-inicial*
  12) Resolvendo o conflito
      - *Pesquise o marcador de conflito:* **<<<<<<<**
      - *As alterações da branch base estão após a linha* **<<<<<<< *HEAD***
      - *o marcador* **=======** *divide as alterações das duas branches*
      - *após o marcador* **>>>>>>>** *está o nome da outra branch*
      - *a resolução do conflito precisa ser* ***commitada***
  13) *git add .*
  14) *commit -m “Resolução do conflito de merge”*
  15) *git checkout atualizacao-pagina-inicial*
  16) *git merge main (atualiza na branch* ***atualizacao-pagina-inicial*** *as últimas mudanças da branch* ***main****)*
  17) *git push origin atualizacao-pagina-inicial*
  18) *git push origin main*

Atualizando seu repositório local com atualizações remotas

1) Acesse a branch ***main*** no *Github*
2) Usando o editor de código do *Github*, faça alguma alteração no arquivo ***index.html*** e, em seguida, o commit
3) Na sua máquina, selecione a branch ***main*** (*git checkout main*)
4) Perceba que o arquivo ***index.html*** está desatualizado em comparação com a versão remota
5) *git pull origin main* (atualiza sua branch ***main*** local com a branch ***main*** remota)

Clonando repositório remoto no seu ambiente local

- Acessar a página do repositório no *Github*, e acessar *Code* => *Clone* => *HTTPS* (copiar a *URL*)
- *git clone <url_https>*
  - Exemplo: git clone https://github.com/ericksharlls/arquivos-oficina-git.git


Dúvidas